Method Parameters

Method Parameters

Definition:

Method parameters are the variables declared in a method definition that receive values (arguments) when the method is called, allowing the method to work on that data.

In Simple Words

Parameters = input variables of a method They help the method accept data from outside.

1.What Exactly happens when you pass parameters?

When you call a method:-

add(10, 20);

Java does 3 steps internally:

1. Create new variables (parameters) in memory
2. Copy values of arguments into those variables
3. Execute method using those copies

Example

```
void add(int a, int b) {
   a = a + 10;
   b = b + 20;
}

int x = 5;
int y = 10;
add(x, y);
```

Internally:

a = 5 (copy of x)
b = 10 (copy of y)

Original x and y stay **Two Main Memory Areas in Java**

2. Memory Level Understanding

Two Main Memory Areas in Java :-

When your program runs, memory is divided into:

1. Stack Memory

- Stores:
  - Method calls
  - Parameters
  - Local variables
- Works like a **stack (LIFO)**
- Very fast ⚡

2. Heap Memory

- Stores:
  - Objects
- Shared across methods

Flow of Method Call :

Let's take this example:

```
class Demo {

  void add(int a, int b) {
    int sum = a + b;
    System.out.println(sum);
  }

  public static void main(String[] args) {
    int x = 10;
    int y = 20;

    Demo d = new Demo();
```

```
d.add(x, y);
    }
}
```

Step-by-Step Memory Execution

Step 1: Main Method Starts

Stack:

main()
x = 10
y = 20
d → reference

Heap:

Demo object created

Step 2: Method Call Happens

d.add(x, y);

New stack frame is created

Step 3: Parameters Created

Stack:

add()
a = 10 (copy of x)
b = 20 (copy of y)
sum = 30

IMPORTANT:

- a and b are **copies**
- Not original variables

Step 4: Method Ends

add() stack frame is removed

Stack becomes:

main()
x = 10
y = 20

a, b, sum → destroyed ❌

3.Types of Method Parameters in Java : -

1. Primitive Type Parameters

Definition:
Parameters that use primitive data types like int, float, char, etc.

Example

```
void show(int x) {
   x = x + 10;
   System.out.println(x);
}

int a = 5;
show(a);
System.out.println(a);
```

Output:

```
15
5
```

Key Point

- Only value is passed (copy)
- Original variable does NOT change

2. Non-Primitive (Reference) Parameters

Definition:
Parameters that use objects, arrays, or classes

Example

```
class Test {
   int x = 10;
}

void change(Test t) {
   t.x = 100;
}

Test obj = new Test();
change(obj);
System.out.println(obj.x); Output:

100
```

Key Point

- Reference copy is passed
- Both refer to same object
- Changes reflect outside

3. Array Parameters

Arrays are also reference type

Example

```
void modify(int[] arr) {
   arr[0] = 99;
}

int[] a = {1, 2, 3};
modify(a);
System.out.println(a[0]);
```

👉 Output:

```
99
```

4.Method Parameter Rules in Java

These are the rules Java follows when you define and pass parameters in methods.

1. Number of Parameters Must Match

Method definition and method call must have the same number of values

Correct

```
void add(int a, int b) { }

add(10, 20);
```

Wrong

```
add(10);        // less arguments
add(10, 20, 30);  // more arguments
```

2.Data Type Must Match

Argument type must match parameter type

Correct

```
void show(int a) { }

show(10);
```

Wrong

```
show("Hello");  // String ≠ int
```

3. Order of Parameters Matters

Java matches values based on position

```
void display(String name, int age)
```

Correct

```
display("Omii", 21);
```

Wrong

```
display(21, "Omii");  // order wrong
```

4. Variable Names Can Be Anything

Parameter names don't need to match argument names

```
void test(int x)

int a = 10;
test(a);
```

x and a are different but it works

5. Parameters Are Local Variables

Parameters exist only inside method

```
void show(int x) {
   System.out.println(x);
}
```

x cannot be used outside method

6. No Duplicate Parameter Names

```
void test(int a, int a)  ERROR
```

Each parameter must have unique name

7. Final Parameters Rule

```
void test(final int x) {
   // x = 20;      not allowed
}
```

You cannot change value of final parameter

8. Type Conversion

```
void test(double x)

test(10);  // int → double (allowed)
```

Smaller → bigger type allowed

9. Varargs Rule

void show(int... nums)

Rules:

- Only **one varargs allowed**
- Must be **last parameter**

Correct

void test(int a, int... b)

Wrong

void test(int... a, int b)

10. Method Overloading Depends on Parameters

Java differentiates methods using parameters

void add(int a, int b)
void add(int a)
void add(double a, double b)

Based on:

- Number
- Type
- Order

11. Default Values Not Allowed

Java does NOT support default parameters like some languages

void test(int a = 10) ERROR

12. Parameters Must Have Data Type

void test(a, b) ERROR

Must specify type:

void test(int a, int b)

5. Parameter vs Local Variables :

Feature	Parameter	Local Variable
Where declared	Method signature	Inside method body
Purpose	Receive input	Store temporary data
Value source	From method call	Assigned inside method
Scope	Inside method	Inside method
Memory	Stack	Stack

Example

```
class Demo {

   void test(int x) {   // parameter
      int y = 20;      // local variable

      System.out.println(x + y);
   }
}
```

Here:

- x → parameter (comes from outside)
- y → local (created inside)

6. Varargs Rule :-

What is Varargs?

Varargs allows a method to accept a variable number of arguments (inputs).

Instead of writing multiple methods, you can use one flexible method.

Syntax

```
void methodName(int... nums) {
   // nums acts like an array
}
```

int... nums = Varargs parameter

Example (Basic)

```
class Demo {

   void add(int... numbers) {
      int sum = 0;

      for (int n : numbers) {
         sum += n;
      }

      System.out.println(sum);
   }

   public static void main(String[] args) {
      Demo d = new Demo();

      d.add(10, 20);       // 2 arguments
      d.add(1, 2, 3, 4);   // 4 arguments
      d.add();             // 0 arguments
   }
}
```